



Prueba de desempeño – Módulo 5.2 Node.js

Caso de uso

La empresa **RiwiMediCare Plus**, dedicada a la distribución de medicamentos e insumos médicos, desea implementar un sistema para gestionar las solicitudes de abastecimiento realizadas por sus diferentes clínicas y centros de atención.

Actualmente, las solicitudes son registradas mediante correos electrónicos y hojas de cálculo, ocasionando pérdida de información, errores en el inventario, demoras en la aprobación de solicitudes y poca trazabilidad sobre el estado de cada requerimiento.

Objetivo

Bajo tu rol como **Desarrollador Backend**, deberás construir una **API REST** utilizando **Node.js**, **Express**, **TypeScript**, **Sequelize**, **PostgreSQL** y **JSON Web Token (JWT)**, que permita administrar el ciclo de vida de las solicitudes de abastecimiento de medicamentos.

El sistema deberá permitir:

- Registrar clínicas y sus responsables.
- Administrar el inventario de medicamentos disponibles en los almacenes.
- Crear solicitudes de abastecimiento.
- Asignar las solicitudes a un almacén.
- Controlar el estado de cada solicitud.
- Consultar el historial de solicitudes realizadas por cada clínica.

Además, la base de datos deberá ser poblada mediante un Endpoint utilizando multer que permita cargar archivos en formato JSON de tal forma actúe como un **Seeders**, con el fin de facilitar la carga de información base que permita efectuar las pruebas y validaciones de la solución.



Funcionalidades principales

Para alcanzar un resultado óptimo de la solución, deberás cumplir con los siguientes requisitos.

Requisitos

Sistema de autenticación

- Registro de usuarios con dos roles (Endpoint sin restricción):
 - Administrador
 - Gestor de Solicitudes
- Inicio de sesión para usuarios registrados.
- Protección de rutas mediante **JSON Web Token (JWT)**.

Persistencia de datos

- Uso de PostgreSQL como base de datos relacional.
- Implementación de Sequelize como ORM.
- Población inicial de la base de datos mediante Seeders para:
 - Usuarios
 - Clínicas
 - Almacenes
 - Medicamentos

Validaciones

La API deberá implementar las validaciones necesarias para garantizar la integridad de la información:

- Existencia de la clínica.
- Existencia del medicamento.
- Disponibilidad suficiente del inventario.
- Estados válidos de la solicitud.
- No permitir registros duplicados de clínicas según su NIT.



Documentación

Toda la API deberá estar documentada utilizando **Swagger**.

Gitflow y Branching

Implementar correctamente:

- Conventional Commits.
- Estrategia de ramas:
 - main
 - develop
 - feature/*

Criterios de aceptación

Funcionalidad completa

Los usuarios podrán registrarse e iniciar sesión. Para efectos de la prueba, el mismo usuario podrá establecer el rol con el cual desea registrarse, por lo cual el EndPoint de registro solo debe validar los datos enviados, más no tener un control por JWT.

Dependiendo del rol, tendrán los siguientes permisos:

Administrador

CRUD completo de:

- Clínicas
- Almacenes
- Medicamentos
- Solicitudes

Todos los usuarios autenticados podrán consultar:

- Solicitudes activas.
- Historial de solicitudes por clínica.

La eliminación de información deberá realizarse de manera lógica utilizando estados para marcar un conjunto de datos como eliminado



Gestor de solicitudes

Se deberá implementar un Endpoint que permita a los usuarios con el rol, registrar una solicitud que incluya:

- Clínica solicitante.
- Medicamento.
- Cantidad solicitada.
- Almacén asignado.
- Estado inicial.

Además:

- Un Endpoint para actualizar el estado de una solicitud existente.
- Un Endpoint para consultar el historial completo de solicitudes registradas.

Middlewares (Validaciones)

La aplicación deberá impedir:

- Registrar solicitudes cuando el almacén no tenga inventario de medicamentos suficiente.
- Registrar dos clínicas con el mismo NIT.
- Registrar cantidades solicitadas menores o iguales a cero.
- Actualizar una solicitud a un estado no permitido.

Clean Code

El código deberá cumplir con buenas prácticas de desarrollo:

- Organización por capas (**Opcional**).
- Separación de responsabilidades (**Obligatorio**).
- Uso adecuado de interfaces y tipos de TypeScript (**Obligatorio**).
- Nombres descriptivos en variables, interfaces, clases (**Obligatorio**).
- Eliminación de código duplicado (**Obligatorio**).
- Sin código fuente comentado (**Obligatorio**).



Entrega y documentación

Se deberá cargar a Moodle, un comprimido en formato zip, con el código fuente de la aplicación, excluyendo del mismo la carpeta node_module (**Obligatorio**). Adicional se debe incluir un backup de la base de datos utilizada en formato .sql.

Se considera completa la entrega si se cumple las siguientes condiciones:

- Organización adecuada del proyecto.
- JSDoc-style comments - Documentación del código fuente
- Uso correcto de tipado: Todas las variables, funciones, métodos, clases, propiedades, interfaces; deben tener un tipado asignado de forma correcta.
- Commits descriptivos siguiendo Conventional Commits.
- README completo indicando:
 - Nombre del Coder.
 - Clan.
 - Tecnologías utilizadas.
 - Instructivo de Instalación.
 - Ejemplo de Variables de entorno.
 - Ejemplo de cómo realizar la Ejecución del proyecto.
 - Ejemplo de cómo ejecutar los seeders para cargar los archivos JSON con data de prueba
 - URL del repositorio en github, el cual debe estar público

Lógica de rutas

- Todas las rutas deberán estar protegidas mediante JWT, excluyendo la del registro de usuario.
- Cada Endpoint deberá validar el rol correspondiente para evitar que un usuario con rol Gestor pueda ejecutar acciones del administrador.
- El administrador podrá utilizar todos los Endpoint disponibles
- Ningún usuario sin autenticación podrá acceder a recursos protegidos.

Consideraciones generales

- Utilizar Node.js versión **18 o superior**.
- La solución deberá enfocarse en garantizar la integridad de la información y el correcto funcionamiento de la lógica del negocio.
- El uso de TypeScript es obligatorio.

Puntos Extras (Opcionales)



Despliegue con Docker (5 Puntos)

Construir una imagen Docker mediante un **Dockerfile** que:

- Instale las dependencias.
- Configure las variables de entorno.
- Ejecute la aplicación.

Docker Compose (5 Puntos)

Crear un archivo `docker-compose.yml` que levante:

- Contenedor de la API.
- Contenedor PostgreSQL.
- Volumen para persistencia.
- Red interna entre ambos servicios.

Pruebas Unitarias con Jest (5 puntos)

Implementar mínimo dos pruebas unitarias sobre alguna de las siguientes funcionalidades críticas:

- Creación de solicitud de abastecimiento.
- Consulta de clínica y asociación de responsable.
- Cambio de estado de solicitudes.

Se deberá garantizar una cobertura superior al **40%**, verificable mediante:

- `npm test -- --coverage`

Documentación de Endpoint con Swagger (5 puntos)

- Documentación de todos los endpoints mediante **Swagger JSDoc**, incluyendo:
 - Método HTTP.
 - Ruta.
 - Descripción.
 - Parámetros.
 - Cuerpo de la solicitud (Request Body).
 - Códigos de respuesta, Ejemplos de petición y respuesta.
- Interfaz de **Swagger UI** configurada y accesible desde el proyecto.
- Documentación completamente funcional y sincronizada con la implementación de la API.

Consideraciones Generales

- Se otorgarán hasta un máximo 20 puntos adicionales sobre la rúbrica de funcionalidad y validaciones a los coder que implementen correctamente los puntos opcionales definidos en la prueba.



- El uso de herramientas de Inteligencia Artificial (IA), incluyendo asistentes, agentes, copilotos de programación están permitidos, haciendo buen uso de estos.
- Se permite la reutilización de lógica, componentes y código propio desarrollados en proyectos realizados durante las diferentes sesiones de entrenamiento de la ruta Node.js.
- Está prohibido compartir el enunciado, los archivos o cualquier contenido relacionado con la prueba de desempeño con otros coders o terceros.
- Durante la ejecución de la prueba no está permitido consultar a otros coders sobre aspectos técnicos, solicitar ayuda para resolver ejercicios o discutir cualquier tema relacionado con la evaluación.
- La prueba se cerrará automáticamente en los siguientes horarios:
 - Jornada AM: 12:59 p. m.
 - Jornada PM: 8:59 p. m.
- Las entregas realizadas después de la hora de cierre serán recibidas con una penalización de 10 puntos sobre la calificación final obtenida.
- El coder es responsable de verificar que la entrega se haya realizado correctamente en la plataforma Moodle.

Recursos:

- [Documentación oficial de JS](#)
- [Documentación oficial de TS](#)
- [Documentación oficial de Node](#)
- [Documentación oficial de Express](#)
- [Documentación oficial de Sequelize](#)
- [Documentación de Json Web Token](#)
- [Documentación oficial Swagger](#)